MEMORIA PROYECTO DISEÑO DE SOFTWARE DOMINIVN2025

Denís Díaz Otero
Víctor Fraga Izquierdo
Xabier Nóvoa Gómez
Adrián Quiroga Linares
Iván Gutiérrez García

Índice

1. Introducción……
2. Fase de inicio…….
    2.1 Modelo de Casos de Uso…….
    2.2 Modelo de Vocabulario……..
3. Fase de elaboración……..
    3.1 Planificación…….
    3.2 Iteración 1…….
    3.3 Iteración 2…….
    3.4 Iteración 3…….
4. Fase de Transición…….

Introducción

La presente memoria documenta el proceso de desarrollo del prototipo "Dominivm 2025", un videojuego retro de estrategia por turnos, desarrollado en el marco del curso de Diseño de Software del GrEI para el curso 2024/25. El objetivo del proyecto es crear una base arquitectónica que permita definir las reglas de enfrentamiento entre imperios de la Edad Antigua (Grecia, Roma, Egipto o Mesopotamia) mediante una interfaz ASCII, sin pretender desarrollar una versión completamente funcional del juego .

Fase de Inicio

Durante esta etapa inicial se sentaron las bases del desarrollo del proyecto, a pesar de haberse reducido a la mitad de la primera sesión de prácticas. Las principales actividades realizadas fueron:

- **Análisis del enunciado:** Se revisaron detalladamente las especificaciones del proyecto, identificando elementos clave como la representación del territorio mediante una cuadrícula, la definición de recursos y construcciones, y la dinámica de interacción entre jugadores.

- **Clarificación de dudas:** Aclaramos todas las dudas con el profesor, lo que permitió interpretar de manera precisa las funcionalidades deseadas y orientar el desarrollo del prototipo conforme a los objetivos planteados.

Aunque fue breve, esta fase ayudó a construir una buena base para el proyecto, uniendo la idea principal con las instrucciones del enunciado y revisándolas con el profesor.

Fase de Elaboración

En esta etapa se concreta el análisis de los requerimientos del sistema, sentando las bases para el desarrollo del prototipo "Dominivm 2025". La Fase de Elaboración se centra en transformar las ideas iniciales y las especificaciones del proyecto en modelos concretos que permitan visualizar el funcionamiento del sistema. Para ello, se han definido dos modelos fundamentales:

- **Modelo de Casos de Uso:** Este modelo identifica y delimita las interacciones entre los usuarios y el sistema, sirviendo como contrato entre el cliente y el equipo de desarrollo. Permite establecer la frontera del sistema y detallar las funcionalidades esenciales que deberá cumplir el prototipo.

- **Modelo de Vocabulario:** Este modelo captura los conceptos clave del proyecto y sus relaciones. En el caso de Dominivm, se definen los elementos y recursos del videojuego, así como las reglas que rigen la interacción entre ellos.

MODELO DE CASOS DE USO

El modelo de casos de uso de *Dominivm 2025* tiene como objetivo capturar de manera clara las principales interacciones entre el **jugador** y el sistema. A continuación, se describen los casos de uso identificados a partir del diagrama adjunto, agrupados por categorías funcionales. Esta visión global permite establecer los requisitos funcionales y delimitar la frontera entre el sistema y su entorno.

A. Configuración y Gestión de Partida

Crear Imperio / Crear Imperios Existentes

- **Objetivo:** Crear un nuevo imperio desde cero o seleccionar uno preconfigurado.
- **Descripción:** El jugador escribe el nombre de un nuevo imperio, o elige uno ya existente con parámetros preestablecidos.

Empezar Partida

- **Objetivo:** Iniciar una sesión de juego con los imperios ya creados y el tablero generado.
- **Descripción:** Se define el punto de partida con imperios posicionados y se da paso a la dinámica de turnos.

Generar Tablero

- **Objetivo:** Crear el tablero de juego.
- **Descripción:** Se genera un mapa con una disposición inicial de celdas.

Dibujar Tablero

- **Objetivo:** Mostrar el tablero de juego con las celdas visibles para dicho imperio.
- **Descripción:** Se genera un mapa con una disposición inicial de celdas, y se dibuja visualmente para los jugadores.

Guardar Partida

- **Objetivo:** Guardar el estado actual del juego.
- **Descripción:** Se almacenan posiciones, estados de unidades, construcciones y recursos para continuar más adelante.

B. Exploración y Visualización

Describir (Mostrar Información de Celda)

- **Objetivo:** Obtener información detallada de una celda.
- **Descripción:** Al seleccionar una celda se muestra contenido con la descripción de su contenido que puede ser: personajes, recursos y construcciones.

Listar Jugadores y Construcciones

- **Objetivo:** Presentar una vista general de los elementos del juego.
- **Descripción:** Incluye datos como ubicación, estado y propietario de unidades y construcciones.

C. Acciones sobre el Terreno y las Unidades

Mover (Mover Personaje)

- **Objetivo:** Desplazar unidades entre celdas.
- **Descripción:** Se valida la celda destino y se actualiza la posición de la unidad o grupo.

Atacar

- **Objetivo:** Realizar ataques entre unidades o construcciones.
- **Descripción:** Se calcula daño, defensa y posibles respuestas. También se contempla el contraataque (por ejemplo en el caso de las torres).

Torre Atacar

- **Objetivo:** Realizar el ataque automático de la torre.
- **Descripción:** Cuando un enemigo entra en una celda vecina o realiza acciones dentro de una de ellas la torre le causa daño.

Recolectar Recursos

- **Objetivo:** Extraer recursos del entorno.
- **Descripción:** Unidades civiles recogen recursos y deben transportarlos posteriormente.

Entrar Construcción

- **Objetivo:** Ingresar a una edificación propia.
- **Descripción:** Otorga beneficios defensivos y/o curación, dependiendo del tipo de construcción.

Agrupar Personajes / Disolver Agrupaciones

- **Objetivo:** Unir o separar unidades en una celda.
- **Descripción:** Permite manejar grupos con capacidades combinadas o separarlos para acciones individuales.

Eliminar Imperio

- **Objetivo:** Quitar del juego a un imperio derrotado.
- **Descripción:** Se elimina cuando se cumplen condiciones como pérdida total de sus aldeas.

D. Construcciones y Gestión de Recursos

Construir

- **Objetivo:** Crear nuevas edificaciones.
- **Descripción:** Requiere gasto de recursos y unidades civiles que realicen el trabajo; transforma la celda en no transitable.

Reparar Construcción

- **Objetivo:** Restaurar vida a construcciones dañadas.
- **Descripción:** Requiere recursos específicos como piedra o madera.

Generar Personaje (Crear Nuevos Personajes)

- **Objetivo:** Crear civiles o militares.
- **Descripción:** Usa recursos y requiere una edificación funcional para producir personajes.

Almacenar Recursos

- **Objetivo:** Transferir recursos recolectados a una edificación.
- **Descripción:** Civiles llevan los recursos hasta una estructura adecuada y los agregan al inventario del imperio.

E. Flujo de Turnos y Fin de Partida

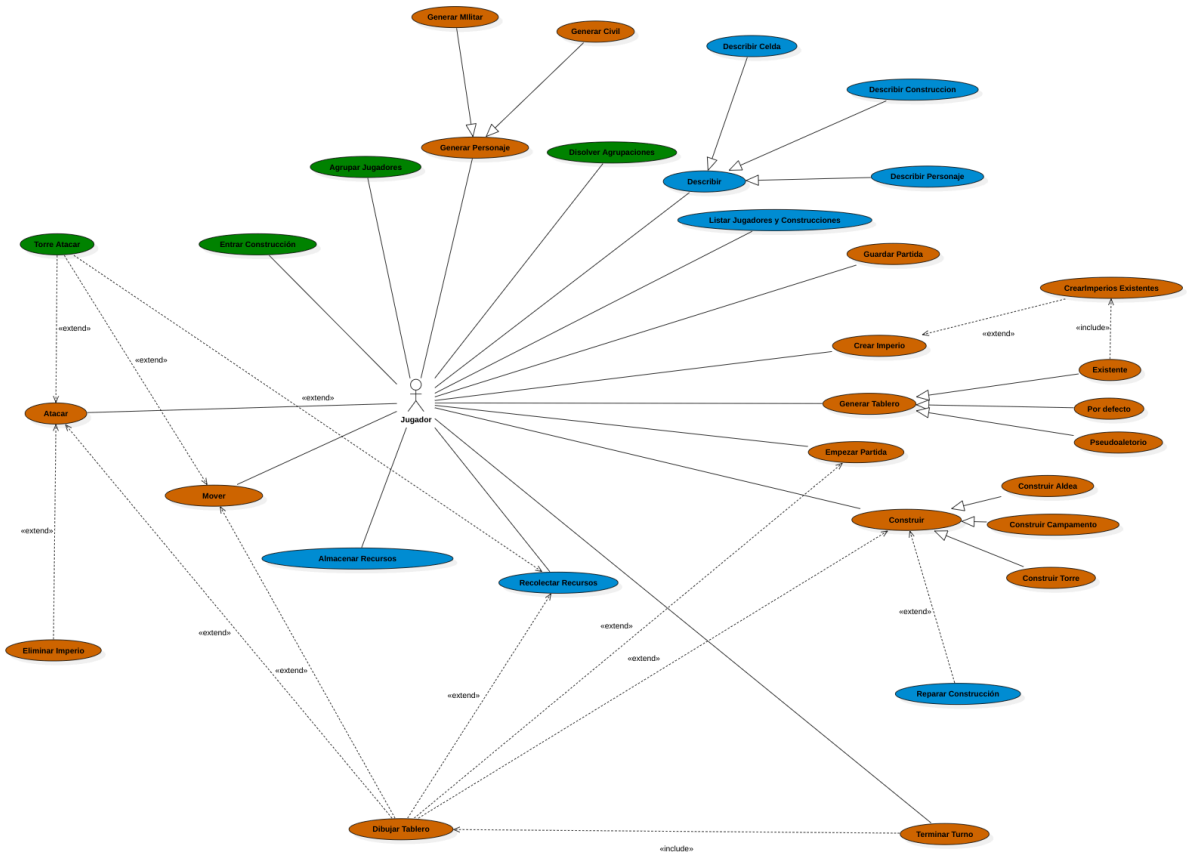
Terminar Turno (Finalizar Turno)

- **Objetivo:** Pasar el control al siguiente jugador.
- **Descripción:** Finaliza las acciones del turno actual y actualiza el estado global para continuar el juego por turnos.

Actores

- **Jugador**: Representa a la persona (o personas) que interactúan con el sistema. En este prototipo, se asume que todos los jugadores utilizan la misma interfaz en turnos alternos. Aun así, cada uno conserva su propio imperio, recursos y unidades.

En la siguiente imagen se puede visualizar el diagrama de casos de uso realizado en la plataforma StarUML



En el siguiente anexo se presenta el modelo de casos de uso más importantes de forma detallada Casos de Uso.

MODELO DE VOCABULARIO

El Modelo de Vocabulario tiene como objetivo principal capturar los conceptos más relevantes de la realidad que se desea soportar, junto con las relaciones existentes entre ellos. Este modelo se centra exclusivamente en la descripción estructural del universo de discurso, sin incluir detalles de implementación o diseño del software. De esta forma, se facilita la comunicación con el cliente, permitiéndole comprender de manera clara y concisa los elementos fundamentales del dominio sin distracciones técnicas.

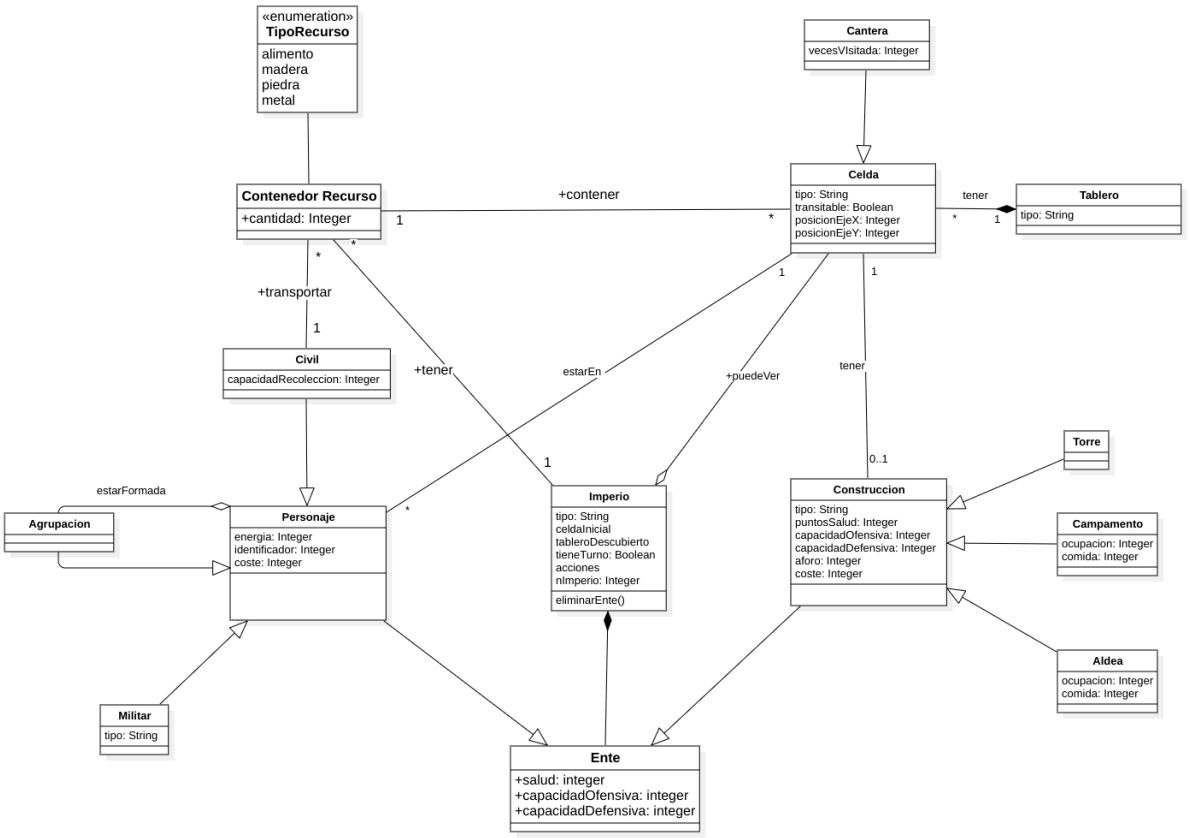
A partir del diagrama del modelo adjunto, se pueden destacar las siguientes observaciones:

- **Conceptos Clave:**
  Se identifican entidades esenciales como *Celda*, *Recurso*, *Construcción* y *Personaje* (donde se distinguen roles como *Civil* y *Militar*). Cada uno de estos conceptos representa un elemento significativo dentro del videojuego, lo que permite estructurar el dominio de forma organizada y coherente.

- **Relaciones entre Entidades:**
  El diagrama ilustra cómo interactúan entre sí estos conceptos. Por ejemplo, una *Celda* puede alojar un *Recurso* o una *Construcción*, y los *Personajes* se desplazan a través de las celdas interactuando con otros elementos. Estas relaciones son fundamentales para entender la dinámica del juego y las interacciones que se darán durante su ejecución.

- **Distinción entre Modelo Conceptual y Diseño del Sistema:**
  Es importante resaltar que este modelo no representa el software en sí, sino una abstracción de los elementos y relaciones existentes en el dominio del problema. Por tanto, no se aborda ningún aspecto relacionado con la implementación o la arquitectura del sistema, sino únicamente la descripción de los conceptos fundamentales y sus interacciones.

La captura del diagrama del Modelo de Vocabulario que se adjunta como parte de este informe es fundamental para visualizar estos conceptos y relaciones. Este recurso permite a todos los clientes disponer de una referencia común, facilitando la discusión y el acuerdo sobre los elementos básicos que constituirán el prototipo del videojuego Dominivm.



En síntesis, el Modelo de Vocabulario es una herramienta clave para garantizar una comprensión compartida del dominio del problema, sirviendo como base para las etapas posteriores del desarrollo del proyecto y asegurando que tanto el equipo técnico como el cliente se encuentren alineados en cuanto a los conceptos y relaciones que definen el universo del juego.

Fase de elaboración

En esta etapa se da inicio al proceso de diseño y se sienta la base para el desarrollo del prototipo mediante la aplicación de un ciclo de vida iterativo e incremental. Abandonando el enfoque lineal del ciclo en cascada, se opta por descomponer el proyecto en iteraciones, cada una de las cuales aborda una parte específica de la funcionalidad. Esta estrategia no solo permite gestionar la complejidad del sistema, sino que también facilita la adaptación a cambios y mejoras continuas a lo largo del desarrollo.

La planificación, que conecta la Fase de Elaboración con la Fase de Construcción, se fundamenta en el Modelo de Casos de Uso, orientando la estructura y el comportamiento esperado del sistema. En cada iteración se definirá y levantará la estructura mínima necesaria, complementada con diagramas de clases y de secuencia que ilustran tanto el diseño en tiempo de compilación como el comportamiento en tiempo de ejecución. Además, se documentará de forma concisa el propósito de cada clase y los patrones de diseño aplicados, aspectos esenciales para lograr una implementación coherente y robusta.

Esta fase representa el momento en que la visión conceptual del proyecto se traduce en una estructura tangible, proporcionando al equipo de desarrollo y al cliente una comprensión clara del avance y las decisiones tomadas en cada iteración.

Planificación

La fase de Planificación organiza el desarrollo de **Dominivm** en iteraciones, cada una con un objetivo específico y un conjunto de casos de uso que guían las tareas de diseño e implementación. A continuación, se describen las tres iteraciones principales establecidas:

Iteración 1 – “MPV (Mínimo producto viable)”

- **Fechas:** 11/03 – 07/04
- **Objetivo:** Desarrollar la funcionalidad mínima que permita iniciar y probar el juego de forma básica, abarcando la configuración de la partida y las primeras acciones esenciales de interacción en el mapa.
- **Casos de uso incluidos:**
    - Crear Imperio / Crear Imperios Existentes: Establecer y registrar un imperio con su jugador correspondiente.
    - Generar Tablero: Crear el mapa de juego inicial con una de las tres configuraciones posibles: por defecto, pseudoaleatorio o existente.
    - Dibujar Tablero: Visualizar el tablero generado desde la perspectiva de un imperio.
    - Guardar Partida: Registrar el estado actual de la partida para continuarla más adelante.
    - Empezar Partida: Activar la sesión de juego con todos los parámetros establecidos.
    - Mover (Personaje): Desplazar personajes por celdas transitables del tablero.
    - Atacar: Ejecutar un ataque desde un personaje o estructura a un enemigo.

    - Construir: Permitir que unidades civiles construyan edificaciones si tienen recursos suficientes.
    - Recolectar Recursos: Extraer recursos del entorno (alimento, madera, piedra, metal) con personajes capacitados.
    - Generar Personaje: Crear nuevos civiles o militares desde edificaciones habilitadas.
    - Terminar Turno: Ceder el control del juego al siguiente jugador.
    - Eliminar Imperio (o Personaje): Eliminar personajes sin puntos de salud o imperios sin capacidad operativa.

Con esta primera iteración se establecen los pilares del sistema, permitiendo una experiencia de juego completa a nivel básico, con las acciones fundamentales para comenzar, interactuar y progresar dentro del mundo del juego.

Iteración 2 – "Expansión básica"

- **Fechas:** 08/04 – 21/04
- **Objetivo:**Ampliar la funcionalidad del juego, mejorando la gestión de los recursos y con la posibilidad de reparar construcciones.
- **Casos de uso incluidos:**
    - Actualizar Mapa: Refrescar visualmente el tablero tras acciones como ataque, recolección o movimiento.
    - Almacenar Recursos: Permitir que los recursos recolectados sean depositados en construcciones habilitadas como aldeas.
    - Reparar Construcción: Restaurar los puntos de salud de edificaciones dañadas usando recursos.
    - Entrar Construcción: Habilitar el ingreso de personajes a construcciones propias o neutrales, respetando el aforo.
    - Agrupar Personajes: Unir varios personajes en un grupo para operar como unidad (movimiento, ataque, etc.).
    - Disolver Agrupaciones: Separar un grupo en sus personajes individuales para realizar acciones independientes.
    - Describir (Celda, Construcción o Personaje): Mostrar información detallada de elementos seleccionados en el tablero.
    - Listar Jugadores y Construcciones: Visualizar un resumen de todos los imperios activos y sus edificaciones.

Esta segunda iteración profundiza en la jugabilidad, aportando sistemas de administración de recursos, restauración de estructuras y organización de unidades, mejorando significativamente la estrategia y el control dentro del juego.

Iteración 3 – "Expansión Avanzada"

- **Fechas:** 22/05 – 06/05
- **Objetivo:** Completar y perfeccionar las características avanzadas, incorporando mecánicas de ataque automático, gestión detallada de unidades y visualización mejorada de la información de juego.

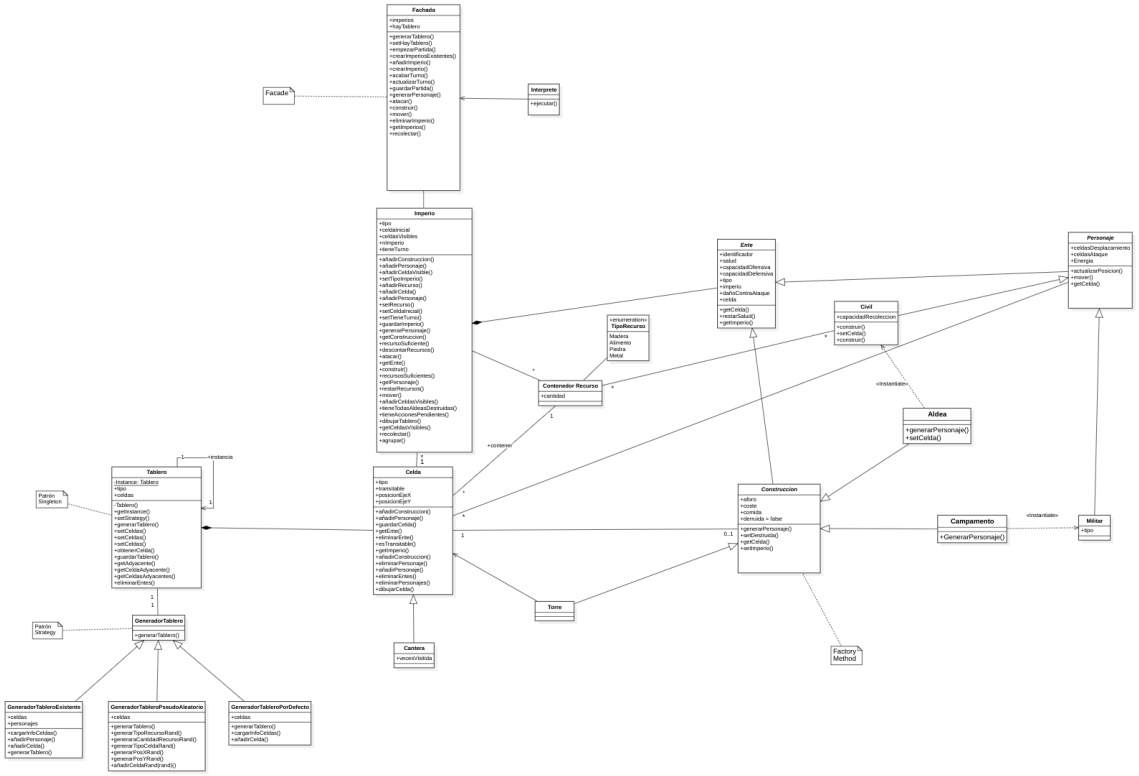
- **Casos de uso incluidos:**
    - Torre Atacar: Las torres atacan automáticamente a enemigos cercanos con su catapulta.
    - Recuperar Salud: Permite a personajes descansar y recuperar salud en construcciones apropiadas.
    - Describir Personaje: Muestra estadísticas y estado detallado del personaje seleccionado.
    - Describir Construcción: Visualiza la descripción y estado de una edificación específica.
    - Describir Celda: Muestra el contenido visible de una celda (construcciones, personajes, recursos).
    - Listar Jugadores: Despliega un resumen de todos los jugadores activos y sus respectivos imperios.
    - Listar Construcciones: Muestra las construcciones activas en el mapa con información de salud y propietario.

En esta última iteración se integran automatismos y mejoras avanzadas que enriquecen la experiencia del jugador, elevando la complejidad táctica y la inmersión mediante una interfaz descriptiva y acciones autónomas en el campo de batalla. En este enlace se puede acceder al documento de la prioridad de los casos de uso.

Iteración 1

Como se ha mencionado anteriormente, el objetivo de esta primera iteración es desarrollar la funcionalidad mínima que permita iniciar y probar el juego de forma básica, abarcando la configuración de la partida y las primeras acciones esenciales de interacción en el mapa.

Durante esta iteración se ha levantado la parte estructural esencial del sistema mediante la creación de clases que representan los conceptos fundamentales del dominio. A continuación, se muestra el diagrama de clases generado en StarUML:



Cada clase tiene una función concreta dentro del sistema, a continuación se documenta brevemente el propósito de las clases más relevantes:

Clase	Propósito
Fachada	Es el punto de entrada a la lógica del juego. Implementa el patrón *Facade*, agrupando todas las operaciones disponibles para el jugador: generar tablero, mover, atacar, recolectar, construir, generar personajes, etc. Aísla la complejidad de las entidades internas.
Imperio	Representa un jugador y sus posesiones (personajes, construcciones, recursos).

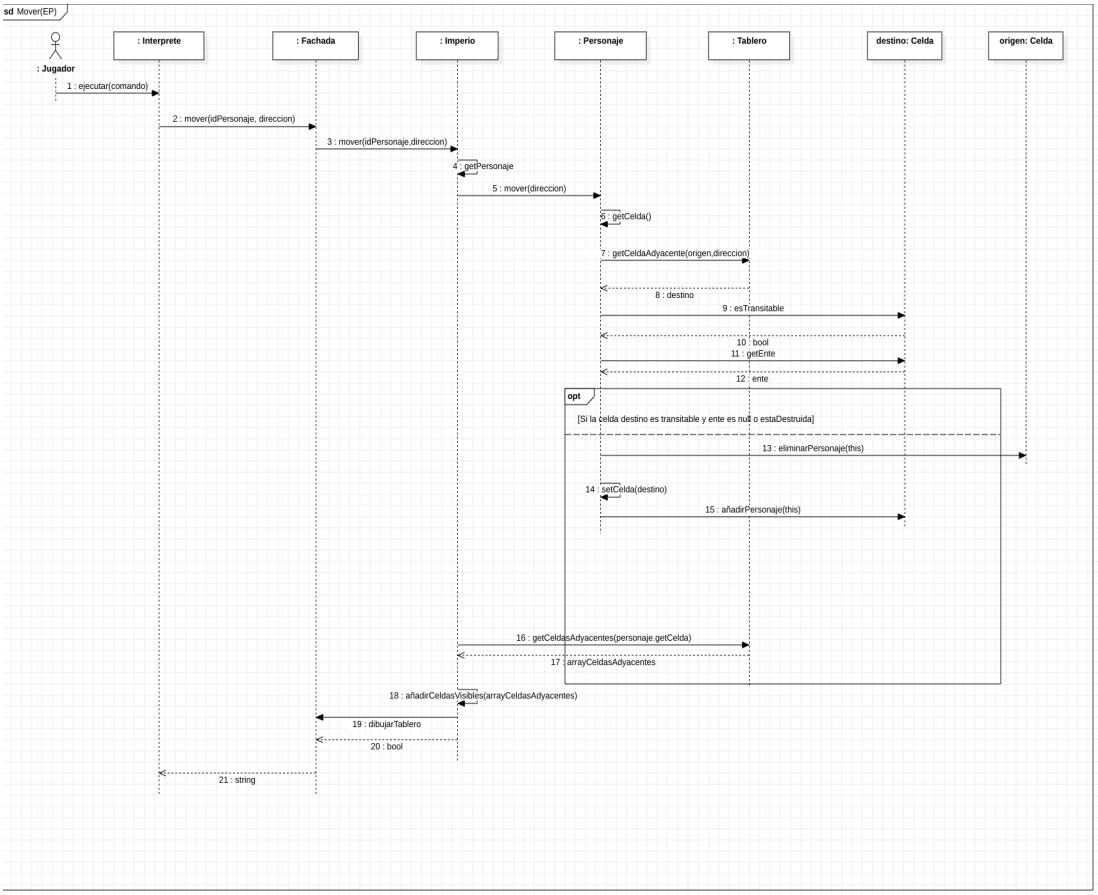
	Contiene métodos para agregar personajes, realizar acciones, gestionar recursos y coordinar la interacción con el tablero.
Tablero	Clase singleton que representa el mapa completo del juego. Contiene las celdas y permite búsquedas, validaciones y actualizaciones sobre la estructura espacial.
Celda	Representa una unidad del tablero. Puede contener personajes, construcciones y recursos. Cada celda mantiene su ocupación actual y posición.
Construcción (clase abstracta)	Base para edificaciones como Aldea, Campamento y Torre. Algunas generan personajes, otras ofrecen defensa o ataque automático.
Personaje (clase abstracta)	Superclase de civil y militar. Posee atributos comunes como posición, salud y capacidades básicas. Define el contrato para comportamientos como moverse o recibir daño.
Civil / Militar	Especializaciones de Personaje. El civil puede construir, recolectar y transportar recursos y el militar puede atacar y participar en combates.
Contenedor Recurso	Modelo para los recursos disponibles en el juego (madera, piedra, alimento, metal), su tipo y cantidad.
GeneradorTablero (y subclases)	Aplica el patrón *Strategy* para ofrecer múltiples formas de generar el tablero: por defecto, pseudoaleatorio o existente.
Intérprete	Componente inicial para el procesamiento de comandos del juego.
Ente (clase abstracta)	Representa a cualquier entidad que ocupa una celda del tablero y participa activamente en el juego, como los personajes (Civil, Militar) y las construcciones (Aldea, Campamento, Torre).

	Es una generalización común que permite tratar por igual a personajes y construcciones en aspectos como posición, salud, o capacidad de recibir daño, sin duplicar código.

En esta iteración se ha implementado parte del comportamiento esperado del sistema, principalmente en torno a la inicialización del juego y la interacción básica entre imperios, personajes y el tablero.

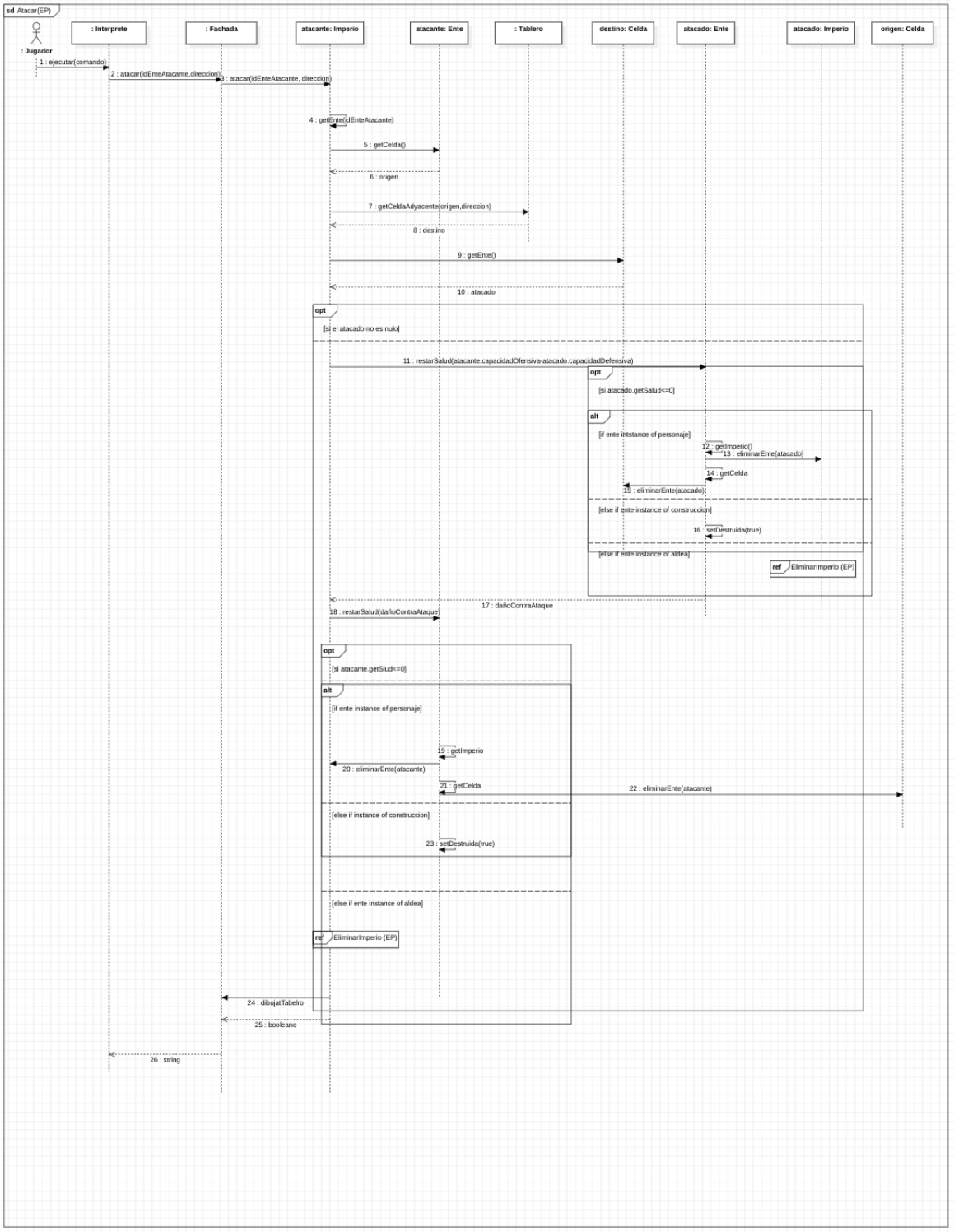
Se han modelado varios diagramas de secuencia que capturan las principales interacciones. A continuación, se describen dos casos destacados por su relevancia en la jugabilidad:

Mover Personaje



1. El jugador selecciona un personaje y una dirección.
2. Se consulta al Tablero si la celda destino es transitable.
3. Se actualiza la posición del personaje y las referencias en las celdas implicadas.
4. Se notifica visualmente el movimiento.

Atacar



1. Un personaje militar ataca a una entidad enemiga.
2. Se verifica la distancia y si el objetivo está en rango.
3. Se calcula el daño y se aplica.
4. Si la salud llega a cero, el objetivo se elimina.

Estos diagramas están disponibles para revisión en StarUML. Solo se incluyen aquí los más significativos.

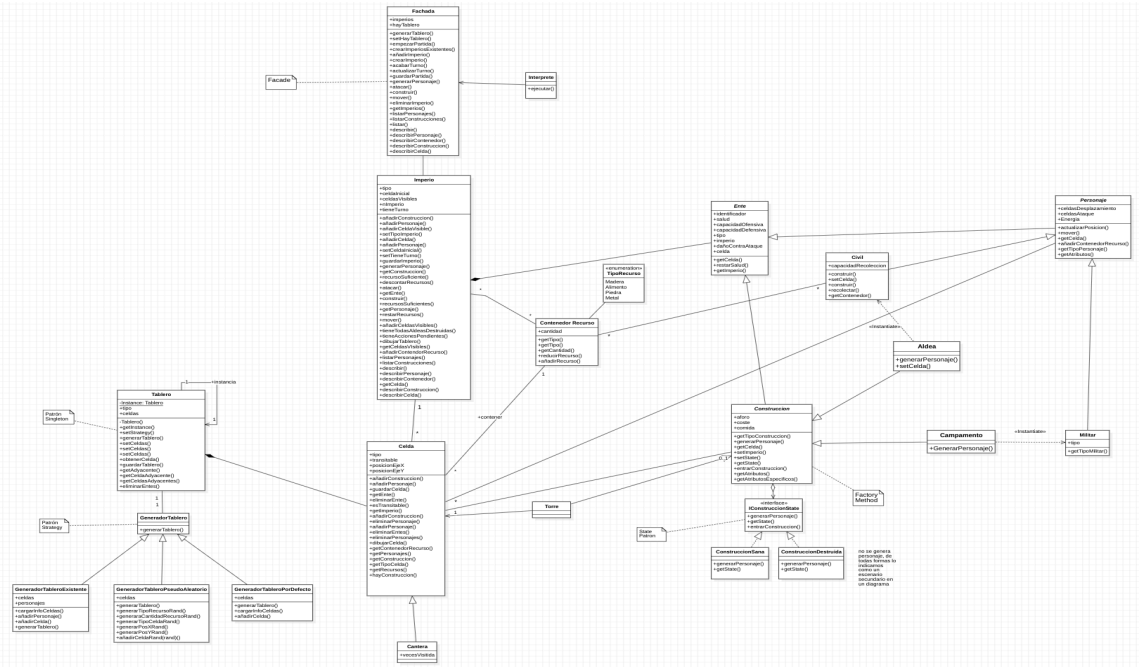
Durante esta primera iteración se han aplicado varios patrones de diseño con el objetivo de establecer una arquitectura flexible y mantenible:

- **Facade**
  Fachada simplifica el acceso al sistema. Los métodos definidos permiten ejecutar acciones del juego sin exponer la lógica interna.
- **Strategy**
  Aplicado en GeneradorTablero, permite usar distintas estrategias de generación del mapa sin modificar el cliente.
- **Singleton**
  Tablero se asegura como una única instancia en todo el ciclo de vida del juego, evitando múltiples representaciones espaciales.
- **Factory Method**
  Las construcciones (Aldea, Campamento) utilizan métodos como generarPersonaje() que podrían evolucionar hacia un patrón *Factory* consolidado.

Estos patrones han sido seleccionados no solo por su adecuación al problema, sino también por su potencial para escalar el diseño conforme evoluciona el desarrollo del proyecto.

Iteración 2

Como ya se explicó anteriormente el objetivo de esta segunda iteración fue ampliar la funcionalidad del juego, mejorando la gestión de recursos, la interacción con las construcciones y la dinámica de agrupación/desagrupación de personajes. Además, se introdujo la capacidad de entrar en construcciones, repararlas, y se refactorizó parte del sistema para prepararlo para futuras expansiones. A continuación, se muestra el diagrama de clases generado en StarUML:



Con respecto a la primera iteración se realizaron los siguientes cambios:

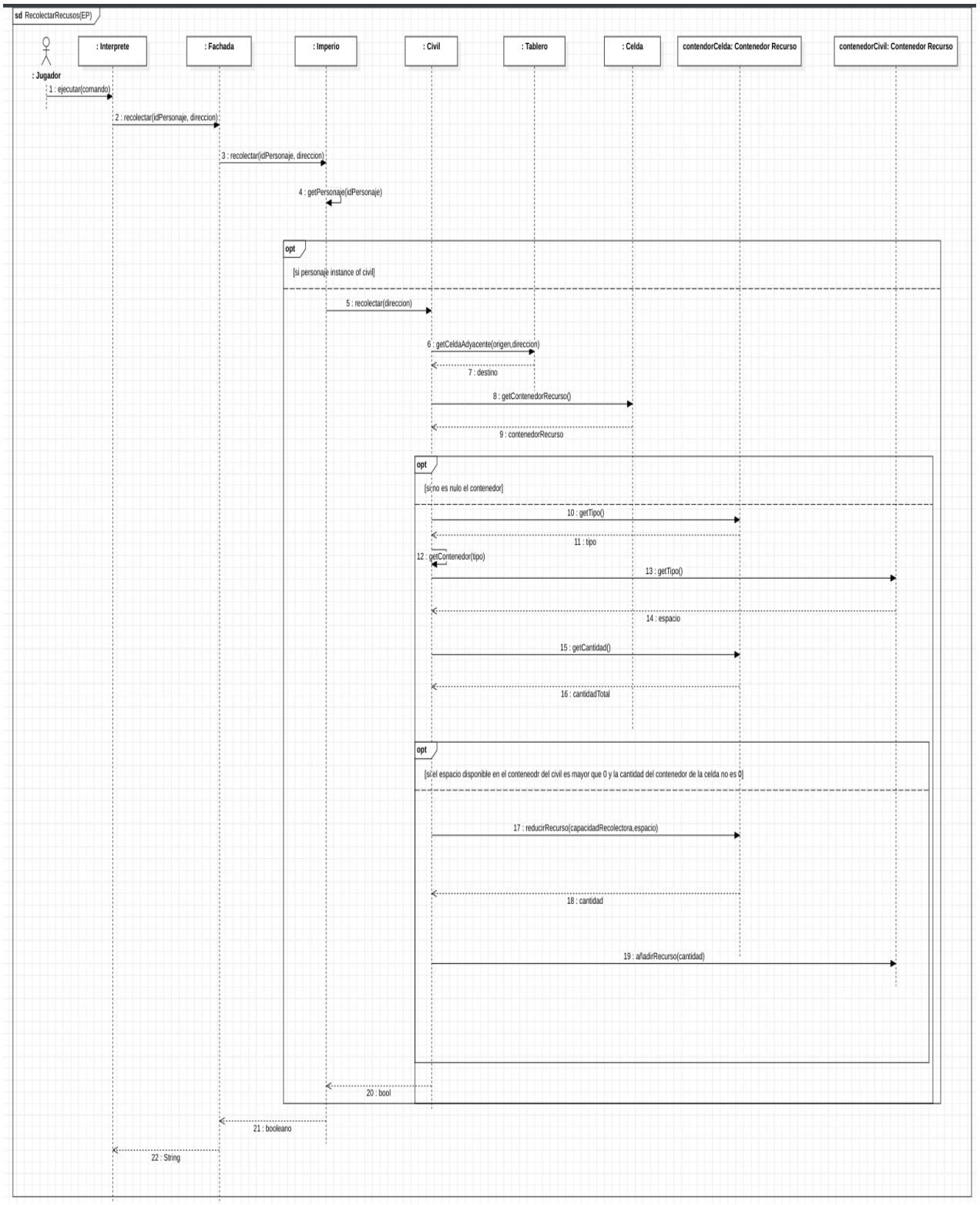
- Se incorporaron interfaces y patrones de estado para representar el estado de las construcciones (IConstruccionEstado, ConstruccionSana, ConstruccionDestruida).
- Se amplió el sistema de construcción con más métodos específicos según estado (puedeAlbergar, generarPersonaje, etc.).
- Se consolidó el uso de Factory Method en Construcción, Campamento, Aldea.
- Se refuerza la validación y reutilización de lógica compartida mediante interfaces y especializaciones controladas.

Las nuevas clases introducidas más relevantes son las siguientes:

IConstruccionEstado	Interface que define el comportamiento de una construcción según su estado (patrón State). Permite separar la lógica de una construcción sana o destruida.
ConstruccionSana	Implementa IConstruccionEstado. Representa una construcción en buen estado que puede generar personajes, almacenar recursos o albergar unidades.
ConstruccionDestruida	Implementa IConstruccionEstado. Representa una construcción inutilizable; no permite ninguna acción funcional.
Torre	Subclase de Construcción. Aunque ya existía como concepto, ahora se consolida en el modelo y se prevé su función de ataque automático en futuras iteraciones.
Cantera	Clase auxiliar asociada a Celda que permite marcar celdas como visitadas por jugadores, útil para visualización y mecánicas de exploración.
ContenedorRecurso	Modela la presencia de recursos (madera, piedra, metal, alimento) en una celda. Esta clase es fundamental para las operaciones de recolección y almacenamiento.
TipoRecurso (enum)	Enumeración que define los tipos de recursos del juego: Madera, Piedra, Alimento, Metal. Mejora la claridad y restringe valores válidos.

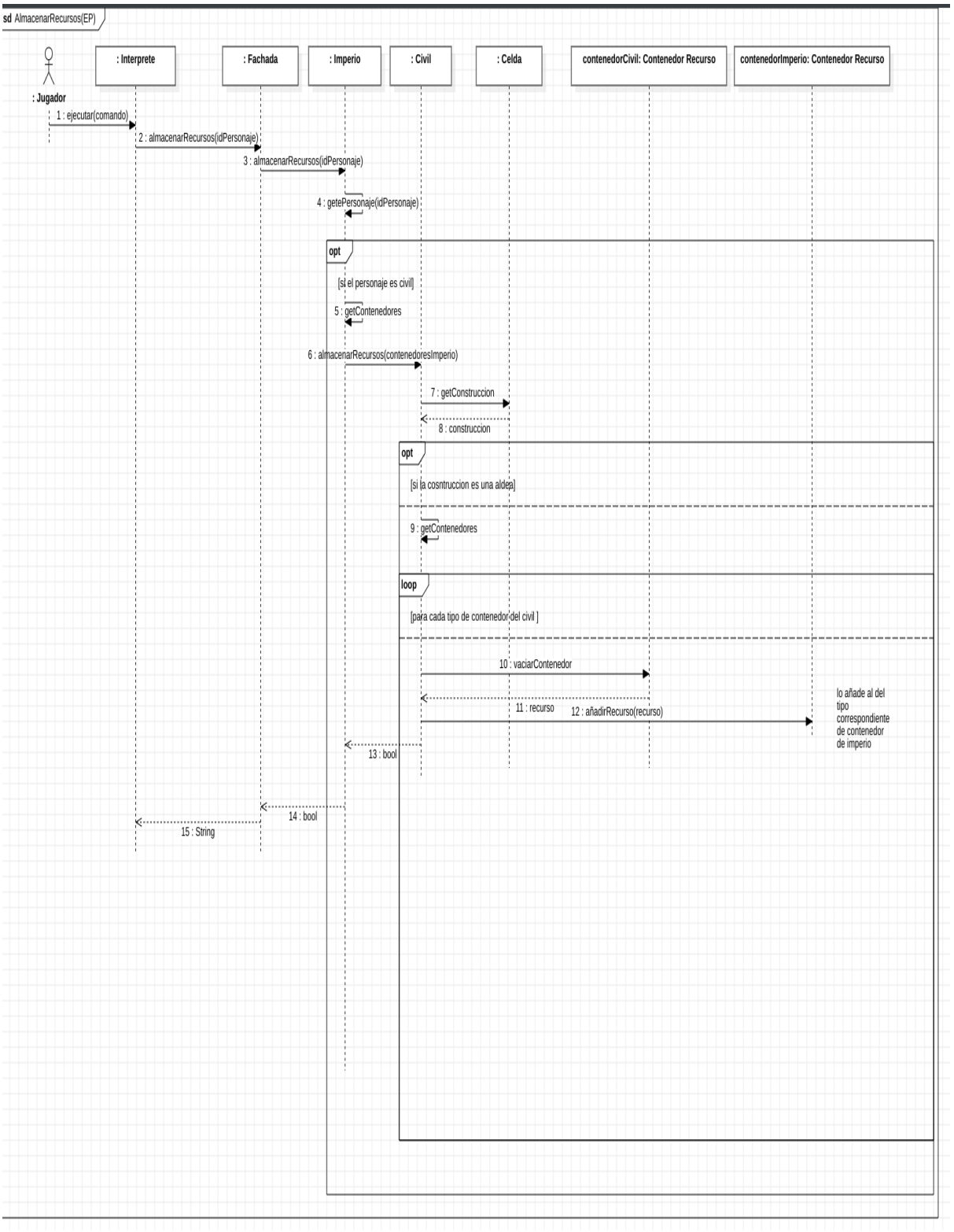
Dentro de esta iteración destacamos los casos de uso que se describen a continuación.

Recolectar Recursos



1. El jugador ordena a un civil recolectar en la celda donde se encuentra.
2. Se valida que haya un ContenedorRecurso con cantidad suficiente.
3. El civil toma una cantidad determinada del recurso.
4. El recurso se añade a su inventario personal o capacidad de carga.

Almacenar Recursos



1. El jugador ejecuta la acción de almacenamiento.
2. Se valida si la construcción permite el almacenamiento (según su tipo y estado).
3. Los recursos del civil se transfieren al Imperio.
4. El inventario del civil se vacía.

En esta segunda iteración se aplicaron nuevos patrones de diseño además de los ya existentes en la primera iteración. Son los siguientes:

- **State**

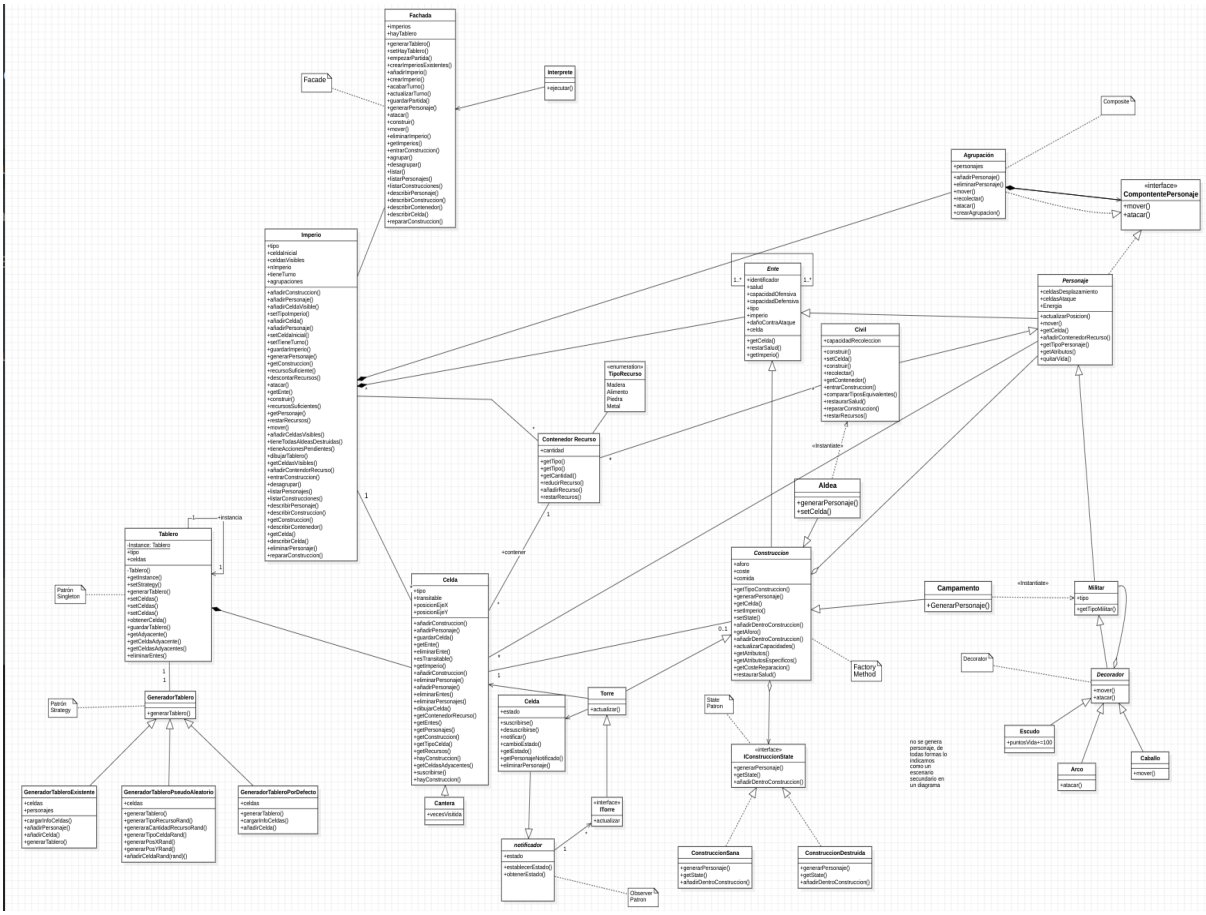
  Aplicado en la gestión del estado de las construcciones (ConstruccionSana, ConstruccionDestruida). Permite que el comportamiento de una construcción cambie dinámicamente según su estado interno.

- **Enum Strategy** (especialización mediante enumeración)

  Se implementa TipoRecurso como un enum para representar tipos de recursos, facilitando su manejo homogéneo y evitando estructuras condicionales repetitivas.

Iteración 3

La tercera iteración del proyecto, “Expansión Avanzada”, tuvo como objetivo completar y perfeccionar las funcionalidades avanzadas del juego, consolidando mecánicas defensivas automáticas, profundizando en la gestión de recursos y permitiendo descripciones detalladas de los elementos del tablero. Con estas mejoras se buscó aumentar la riqueza estratégica de la partida y ofrecer al jugador información completa y contextual para la toma de decisiones. A continuación se muestra una imagen de esta iteración



Con respecto a la anterior iteración se realizaron los siguientes cambios:

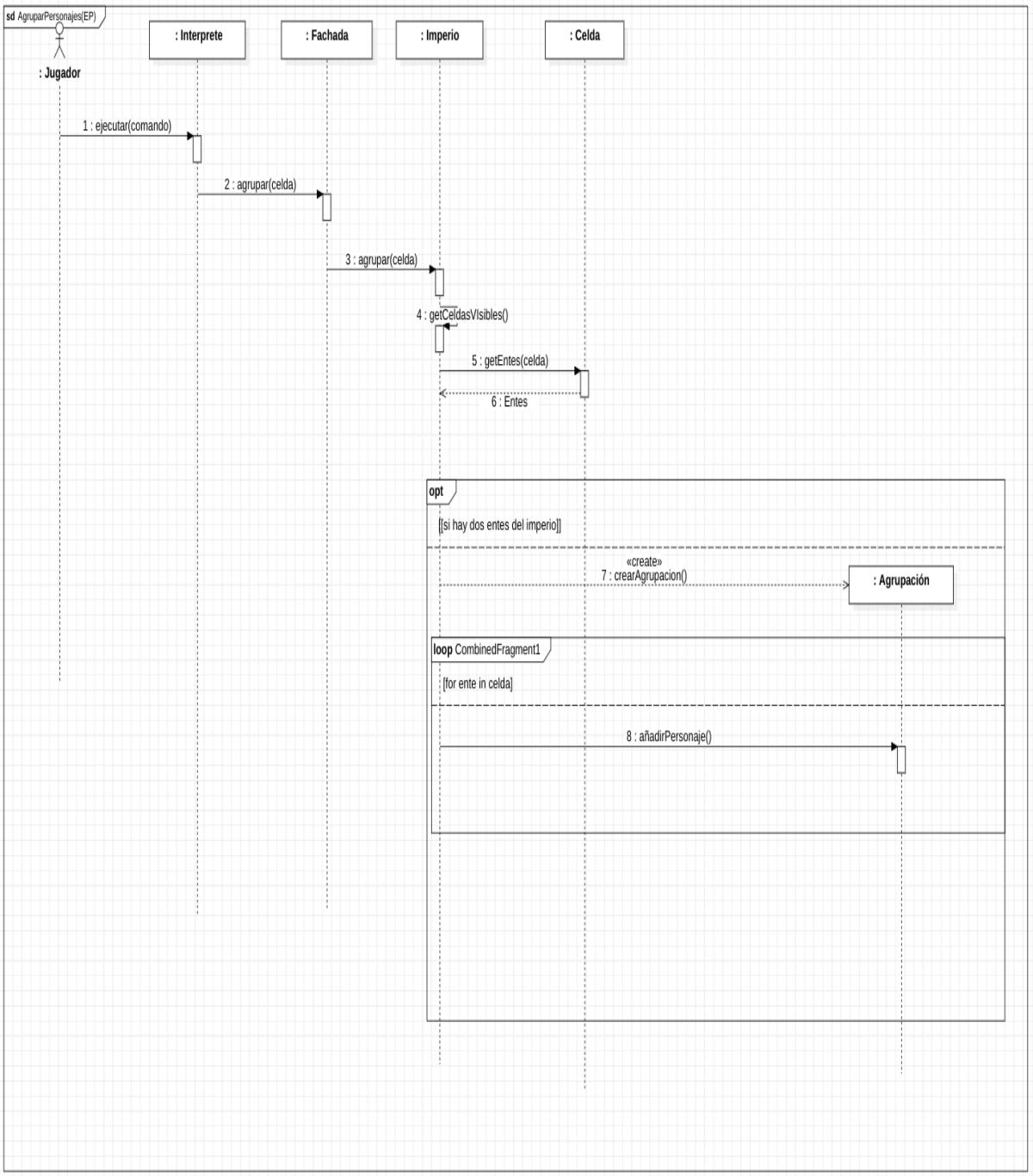
- Se incorporó el patrón Composite en el sistema de agrupación de personajes (Agrupacion y la interfaz ComportamientoPersonaje), permitiendo tratar de forma uniforme a personajes individuales y grupos.
- Se añadieron nuevas especializaciones de personajes militares (como Escudo, Arco, Caballo) usando composición sobre herencia para facilitar combinaciones de equipamiento.
- Se mejoró la organización de Celda, integrando el sistema de ataque desde torres y la interacción avanzada con ContenedorRecurso y Construcción.

Se introdujeron nuevas clases para llevar a cabo estos cambios:

Agrupacion	Implementación del patrón Composite. Permite agrupar múltiples instancias de Personaje bajo una misma entidad lógica que responde a las interfaces mover() y atacar().
ComportamientoPersonaje (Interface)	Define la abstracción que tanto Personaje como Agrupacion implementan.
Escudo, Arco, Caballo	Clases que representan diferentes equipamientos o capacidades ofensivo/defensivas para los personajes militares. Utilizan composición para dotar de comportamiento específico a la clase Militar.

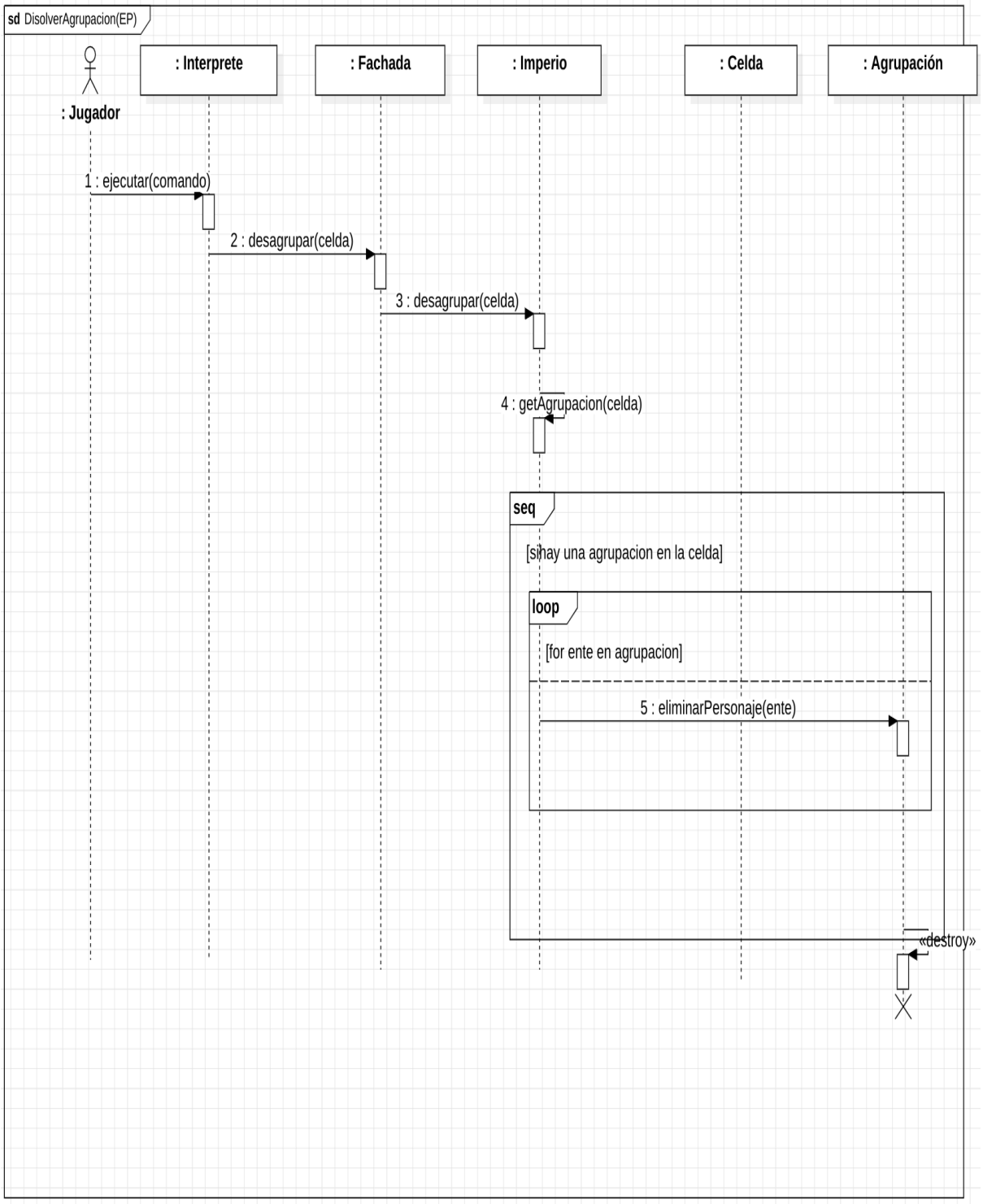
Como caso de uso a destacar en esta iteración podemos tener en cuenta la agrupación y desagrupación de los personajes. Además también es de gran interés el caso de atacar de la torre ya que está implementado con el patrón de diseño Observer.

Agrupar Personajes



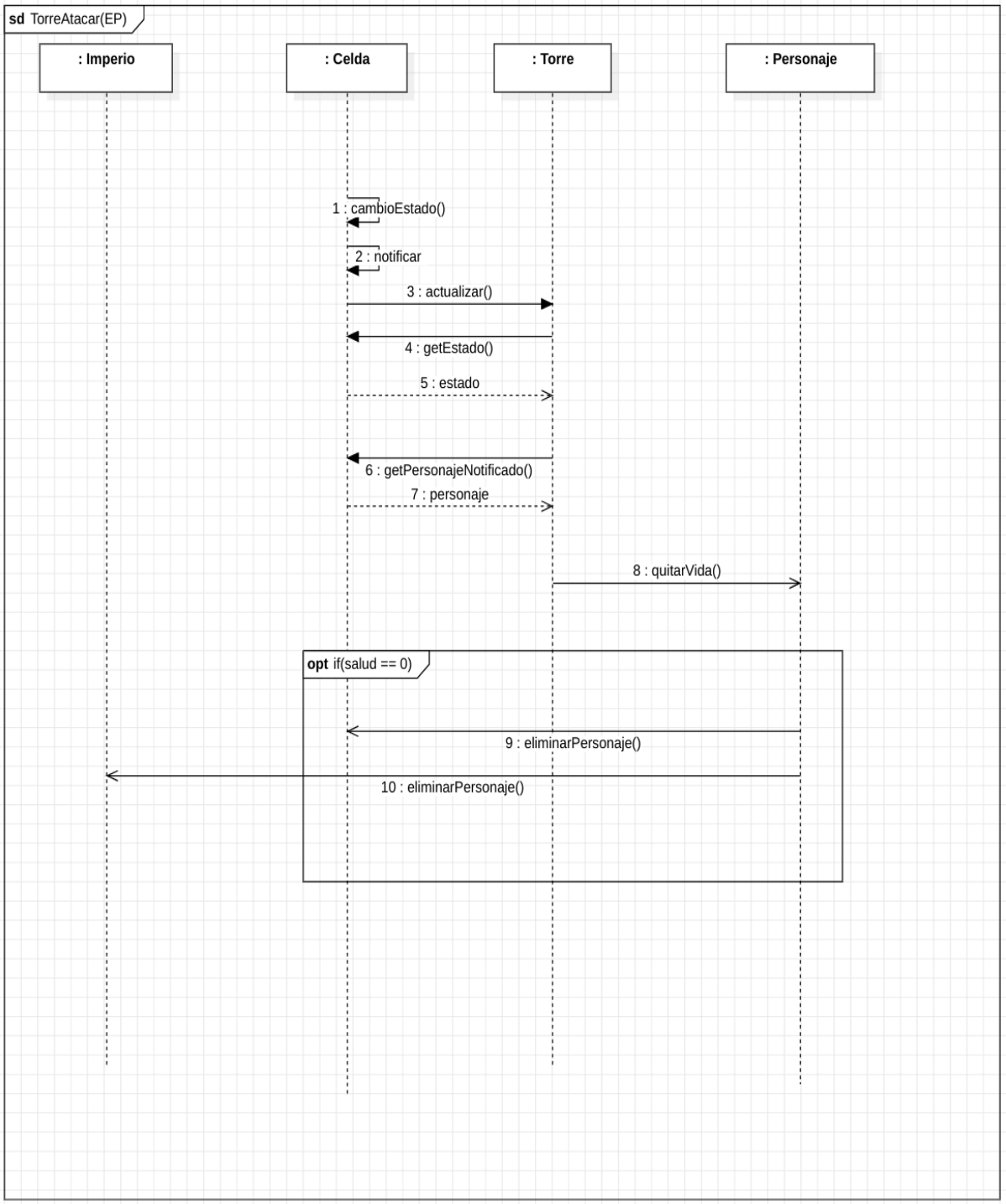
1. El jugador selecciona múltiples personajes sobre el tablero.
2. Se valida que todos los personajes estén en la misma celda y no pertenezcan a otra agrupación.
3. Se crea una instancia de Agrupacion que implementa ComportamientoPersonaje.
4. Cada personaje seleccionado es asociado a la nueva agrupación y deja de estar disponible como entidad individual.

Disolver Agrupaciones



1. El jugador selecciona una agrupación activa.
2. Se ejecuta el comando de disolución desde la Fachada.
3. La agrupación recorre su lista interna de personajes y los desvincula del grupo.
4. Cada personaje recupera su independencia como Personaje individual y vuelve a estar accesible de forma separada.
5. La instancia de Agrupacion es eliminada.

Torre Atacar



1. El Imperio notifica un cambio de estado en una Celda.
2. La Torre, como observador, se actualiza y verifica su estado.
3. Si está funcional, obtiene el personaje notificado en la celda.
4. Ejecuta quitarVida() al personaje.
5. Si su salud llega a cero, se invoca eliminarPersonaje() y se retira del tablero.

Por último, en esta tercera iteración se introdujeron los siguientes patrones de diseño:

- **Observer**

  Aplicado entre Celda y las construcciones como Torre. Permite que las torres reaccionen automáticamente a los cambios en su entorno (por ejemplo, la llegada de enemigos) sin necesidad de intervención directa, promoviendo un sistema reactivo y desacoplado.

- **Decorator**

  Utilizado para extender dinámicamente las capacidades ofensivas de las unidades militares (Militar) sin modificar sus clases base. Esto permite añadir equipo como Arco, Caballo o Escudo que modifica el comportamiento de ataque o defensa de forma flexible.

Fase de Transición

El día 15/05/2025 se entregan los modelos desarrollados a lo largo del proyecto para que sean sometidos a valoración por parte del profesor. Con esta entrega se da por concluido el proceso de diseño, consolidando todas las iteraciones previas y sus respectivas mejoras.